

LABORATORY

**LAB 01**

# Digital Output Fundamentals

GPIO Control with EduFramework

S32K144 · MaaZEDU

EduFramework

**FPT University**

# Contents

|     |                                                            |   |
|-----|------------------------------------------------------------|---|
| 1   | Introduction .....                                         | 2 |
| 1.1 | Lab Overview .....                                         | 2 |
| 1.2 | Objectives .....                                           | 2 |
| 2   | Background .....                                           | 2 |
| 2.1 | Digital Output and Logic States .....                      | 2 |
| 2.2 | Digital Output and Active-Low LEDs .....                   | 2 |
| 2.3 | LED Blinking Cycle .....                                   | 3 |
| 3   | Hardware Setup .....                                       | 3 |
| 3.1 | Required Hardware .....                                    | 3 |
| 3.2 | Onboard LED Mapping .....                                  | 4 |
| 4   | EduFramework API .....                                     | 4 |
| 4.1 | <code>pinMode()</code> .....                               | 4 |
| 4.2 | <code>digitalWrite()</code> .....                          | 4 |
| 4.3 | <code>delay()</code> .....                                 | 5 |
| 5   | Lab Exercise .....                                         | 5 |
| 5.1 | Requirements .....                                         | 5 |
| 5.2 | Program .....                                              | 5 |
| 5.3 | Verification .....                                         | 6 |
| 6   | Extension .....                                            | 6 |
| 6.1 | Toggling the State with <code>digitalToggle()</code> ..... | 6 |
| 6.2 | Timing with <code>millis()</code> .....                    | 7 |
| 6.3 | Extended Exercise .....                                    | 7 |
| 7   | References .....                                           | 8 |

# 1 Introduction

## 1.1 Lab Overview

Digital Input/Output (Digital I/O) is a fundamental GPIO function that allows a microcontroller to exchange logic states with external hardware. When a GPIO pin is configured as an output, the program can set the logic state of the pin to generate a digital signal [1].

This lab is designed to introduce Digital Output through the APIs provided by EduFramework. An onboard LED on the MaaZEDU Development Board is used as a visual output. By alternating between the `HIGH` and `LOW` logic states over time, the program implements a basic LED blinking application.

## 1.2 Objectives

### OBJECTIVES

The objectives of this laboratory are:

1. Explain the basic principle of Digital Output and the meaning of the `HIGH` and `LOW` logic states.
2. Identify the relationship between GPIO logic states and the state of an active-low LED.
3. Use the `pinMode()`, `digitalWrite()`, and `delay()` APIs to control Digital Output.
4. Develop and verify an LED blinking application on the MaaZEDU Development Board.

# 2 Background

## 2.1 Digital Output and Logic States

GPIO (General-Purpose Input/Output) pins can be configured to perform digital input or output functions. When configured as a Digital Output, the logic state of the pin is controlled by the program. In EduFramework, the output mode is represented by `OUTPUT`, while the two logic states are represented by `HIGH` and `LOW`.

| State             | Value           | Meaning           |
|-------------------|-----------------|-------------------|
| <code>LOW</code>  | <code>0U</code> | Low logic state.  |
| <code>HIGH</code> | <code>1U</code> | High logic state. |

`HIGH` and `LOW` represent logic states rather than fixed voltage values. The actual voltage levels and electrical limits of an I/O pin depend on the characteristics of the microcontroller. For the S32K1xx family, these specifications are defined in the NXP Data Sheet [3].

## 2.2 Digital Output and Active-Low LEDs

LEDs are commonly used to visually represent the state of a digital output. However, the relationship between the GPIO logic state and whether the LED is on or off depends on how the LED is connected in the circuit.

| Configuration | LED ON            | LED OFF           |
|---------------|-------------------|-------------------|
| Active-high   | <code>HIGH</code> | <code>LOW</code>  |
| Active-low    | <code>LOW</code>  | <code>HIGH</code> |

The onboard LEDs used in the hardware configuration of this lab operate according to the active-low principle. Therefore, `LOW` turns the LED on, whereas `HIGH` turns the LED off.

#### Note

`HIGH` and `LOW` describe the logic state of a GPIO pin, whereas `ON` and `OFF` describe the state of the connected device. The relationship between these states is determined by the hardware configuration.

### 2.3 LED Blinking Cycle

An LED blinking application periodically changes the state of the LED. In this lab, the LED remains on for `500 ms` and off for `500 ms`. Therefore, the duration of one complete cycle is:

$$T = 500 \text{ ms} + 500 \text{ ms} = 1000 \text{ ms} = 1 \text{ s}$$

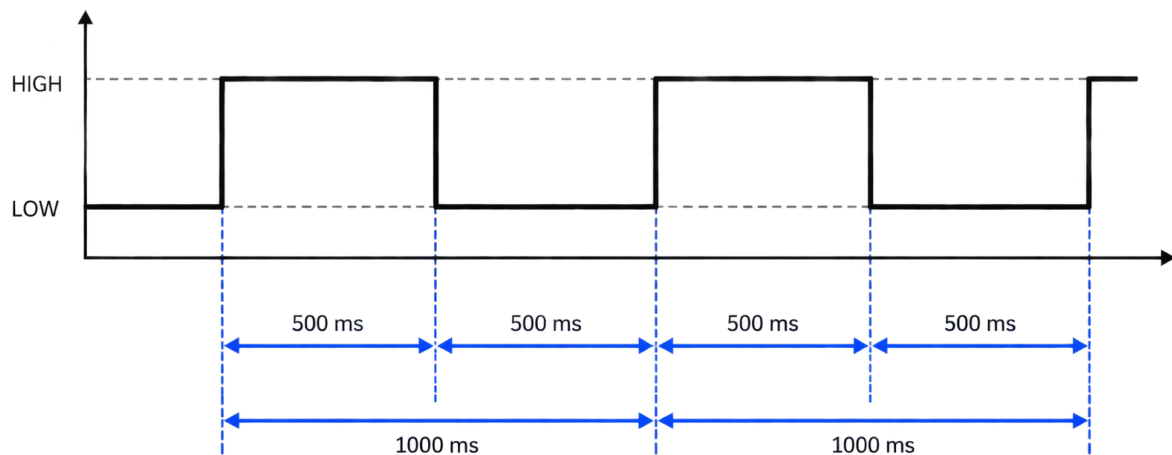


Figure 1.2.1. Timing relationship of the LED blinking application

The interval between state changes is generated using `delay()`, allowing both LED states to be directly observed on the hardware.

## 3 Hardware Setup

### 3.1 Required Hardware

This lab directly uses an onboard LED on the MaaZEDU Development Board and does not require any additional external components or wiring.

| Hardware                  | Description                                                         |
|---------------------------|---------------------------------------------------------------------|
| MaaZEDU Development Board | Development board based on the S32K144 microcontroller.             |
| USB Cable                 | Connects the board to the computer for power and program uploading. |

Table 1.3.1. Hardware used in this lab

## 3.2 Onboard LED Mapping

EduFramework provides Logical Pins for accessing the onboard LEDs from application code. The hardware mapping used by the framework is shown below:

| Component | Logical Pin | MCU Pin | Function       |
|-----------|-------------|---------|----------------|
| Red LED   | LED_RED     | PTD15   | Digital Output |
| Blue LED  | LED_BLUE    | PTD16   | Digital Output |
| Green LED | LED_GREEN   | PTD0    | Digital Output |

Table 1.3.2. Onboard LED mapping on the MaaZEDU Development Board

The main exercise uses `LED_RED`, which is mapped by EduFramework to the S32K144 `PTD15` pin.

## 4 EduFramework API

The LED blinking application uses three APIs: `pinMode()` to configure a Digital Output, `digitalWrite()` to set the logic state, and `delay()` to provide a time interval between state changes. These APIs follow the familiar Arduino programming model [2].

### 4.1 `pinMode()`

`pinMode()` is used to configure the operating mode of a digital pin.

#### `pinMode`

Configures the operating mode of the specified Logical Pin.

#### Syntax

```
pinMode(pin, mode);
```

#### Parameters

| Parameter         | Type / accepted value | Description                                              |
|-------------------|-----------------------|----------------------------------------------------------|
| <code>pin</code>  | Logical Pin           | Pin to be configured, for example <code>LED_RED</code> . |
| <code>mode</code> | <code>OUTPUT</code>   | Configures the pin as a Digital Output.                  |

#### Return value

No return value.

Example:

```
pinMode(LED_RED, OUTPUT);
```

### 4.2 `digitalWrite()`

`digitalWrite()` is used to set the logic state of a Digital Output pin.

#### `digitalWrite`

Sets the logic state of the specified Logical Pin.

#### Syntax

```
digitalWrite(pin, value);
```

**Parameters**

| Parameter | Type / accepted value                | Description                                              |
|-----------|--------------------------------------|----------------------------------------------------------|
| pin       | Logical Pin                          | Pin to be controlled, for example <code>LED_RED</code> . |
| value     | <code>HIGH</code> / <code>LOW</code> | Logic state to be applied to the output pin.             |

**Return value**

No return value.

Example:

```
digitalWrite(LED_RED, LOW);  
digitalWrite(LED_RED, HIGH);
```

### 4.3 delay()

`delay()` creates a waiting period before the program continues with the next instruction.

**delay**

Creates a time delay specified in milliseconds.

**Syntax**

```
delay(ms);
```

**Parameters**

| Parameter | Type / accepted value | Description                              |
|-----------|-----------------------|------------------------------------------|
| ms        | Time                  | Required delay duration in milliseconds. |

**Return value**

No return value.

Example:

```
delay(500U);
```

The statement above creates a delay of `500 ms`.

## 5 Lab Exercise

Create a PlatformIO project using EduFramework for the MaaZEDU Development Board.

### 5.1 Requirements

Develop a program that continuously blinks the onboard red LED. The LED remains on for `500 ms`, then turns off for `500 ms`, and the sequence repeats continuously.

### 5.2 Program

Implement the following program in `src/main.c`:

```
#include "Arduino.h"

int main(void)
{
    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        digitalWrite(LED_RED, LOW);
        delay(500U);

        digitalWrite(LED_RED, HIGH);
        delay(500U);
    }

    return 0;
}
```

*Listing 1.5.1. LED blinking program using EduFramework*

`setup()` initializes EduFramework before its APIs are used. `pinMode()` then configures `LED_RED` as a Digital Output. Inside the `while (1)` loop, `digitalWrite()` alternately sets `LOW` and `HIGH`; each state is maintained for 500 ms using `delay()`.

Because the LED operates in an active-low configuration, `LOW` corresponds to the ON state and `HIGH` corresponds to the OFF state.

### 5.3 Verification

Build and upload the program to the MaaZEDU Development Board, then observe the onboard red LED.

#### EXPECTED RESULT

The red LED remains on for approximately 500 ms, turns off for approximately 500 ms, and repeats continuously. One complete blinking cycle lasts approximately 1 s.

## 6 Extension

In addition to the basic APIs used in the main exercise, EduFramework provides additional and more advanced APIs for a variety of application requirements. In this section, `digitalToggle()` is used to toggle the state of a Digital Output, while `millis()` is used to track elapsed execution time without creating a waiting period with `delay()`.

### 6.1 Toggling the State with `digitalToggle()`

`digitalToggle()` toggles the current logic state of a Digital Output. If the current state is `LOW`, the pin changes to `HIGH`; conversely, if the current state is `HIGH`, the pin changes to `LOW`.

#### **digitalToggle**

Toggles the current logic state of the specified Logical Pin.

#### Syntax

```
digitalToggle(pin);
```

**Parameters**

| Parameter | Type / accepted value | Description                                      |
|-----------|-----------------------|--------------------------------------------------|
| pin       | Logical Pin           | Digital Output pin whose state is to be toggled. |

**Return value**

No return value.

**Extension:** Rewrite the LED blinking program using `digitalToggle()` so that the LED state still changes every `500 ms`.

## 6.2 Timing with `millis()`

`delay()` creates a waiting period and holds the execution flow until the specified duration has elapsed. In applications that need to continue processing other tasks while tracking time, `millis()` can be used to determine the elapsed time without stopping the main loop.

**`millis`**

Reads the number of milliseconds elapsed since the system time base was initialized.

**Syntax**

```
millis();
```

**Return value**

Elapsed time in milliseconds.

A time interval can be checked using the following principle:

```
if ((millis() - previousTime) >= interval)
{
    /* Periodic operation */
}
```

*Listing 1.6.1. Checking a time interval using `millis()`*

Here, `previousTime` stores the timestamp of the previous operation, while `interval` defines the time interval between two operations.

**Extension:** Rewrite the LED blinking application with a `500 ms` state-change interval without using `delay()`. When the required interval has elapsed, use `digitalToggle()` to change the LED state and update the timestamp.

## 6.3 Extended Exercise

Develop a program that simultaneously controls the three onboard LEDs with independent state-change intervals:

- `LED_RED : 500 ms`.
- `LED_BLUE : 1000 ms`.
- `LED_GREEN : 1500 ms`.

Do not use `delay()` inside the main loop. Each LED must use an independent timestamp so that timing one LED does not stop the processing of the other LEDs.

---

### EXPECTED RESULT

The three LEDs blink simultaneously according to their respective time intervals and operate independently in terms of timing.

## 7 References

---

- [1]     Arduino, *Digital Pins* , Arduino Documentation.
- [2]     Arduino, *Arduino Language Reference* , Arduino Documentation.
- [3]     NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.
- [4]     MaaZEDU, *MaaZEDU Development Board Guide* .